

List Topologies

SINGLY, DOUBLY, & CIRCULAR STRUCTURES

Data Structures: Module 17

codeHive

1. Singly Linked List

Efficiency King

The simplest form. Each node points only to the **Next** node. It uses the least amount of memory per node but only supports one-way traversal.

`[Data|Next] → [Data|Next] → [Data|NULL]`

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Linking Logic
node1.next = node2
node2.next = node3
```

2. Doubly Linked List

Bidirectional Path

Each node contains two pointers: **Next** and **Previous**. This allows you to traverse both forward and backward, making deletions easier at the cost of higher memory usage.

`[NULL|Data|Next] ⇌ [Prev|Data|Next] ⇌ [Prev|Data|NULL]`

```
class Node:
    def __init__(self, data):
        self.data = data
```

```
self.next = None  
self.prev = None
```

```
# Linking Logic  
node2.prev = node1  
node2.next = node3
```

codeHive

3. Circular Varieties

Infinite Loops

In circular lists, the **Tail** node points back to the **Head** instead of reaching a NULL end. Excellent for tasks that cycle (like CPU scheduling or round-robin buffers).

Circular Singly

[Head] → [Node] → [Tail] → ↪ back to [Head]

```
# Making it circular
node4.next = node1

# Traversal Caution:
while currentNode ≠ startNode:
```

Circular Doubly

The ultimate flexibility. Head's prev points to Tail, and Tail's next points to Head.

```
node1.prev = node4
node4.next = node1
```

4. Selecting the Architecture

Type	Memory	Traversal
Singly	Low (1 ptr/node)	Forward only
Doubly	High (2 ptrs/node)	Forward & Backward
Circular	Varies	Infinite Loop

codeHive

5. Knowledge Check

Challenge: How is a list made circular?

Answer: By connecting the next pointer of the last node to the Head node.

© 2026 codeHive Academy • Advanced Data Structure Topologies

codeHive